

MeldableHeap

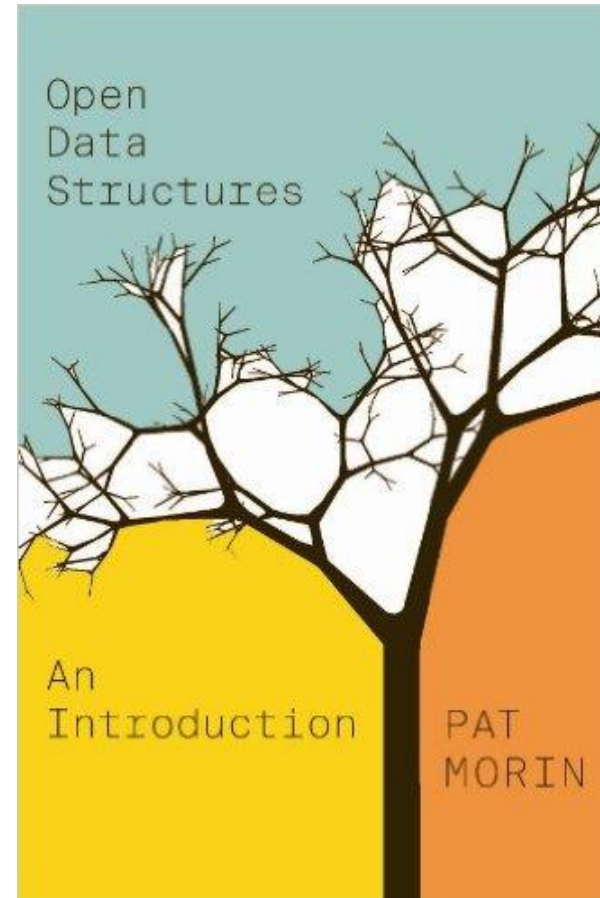
COMP 2402

Abstract Data Types & Algorithms

Readings

Meldable Heap

- Chapter 10.2



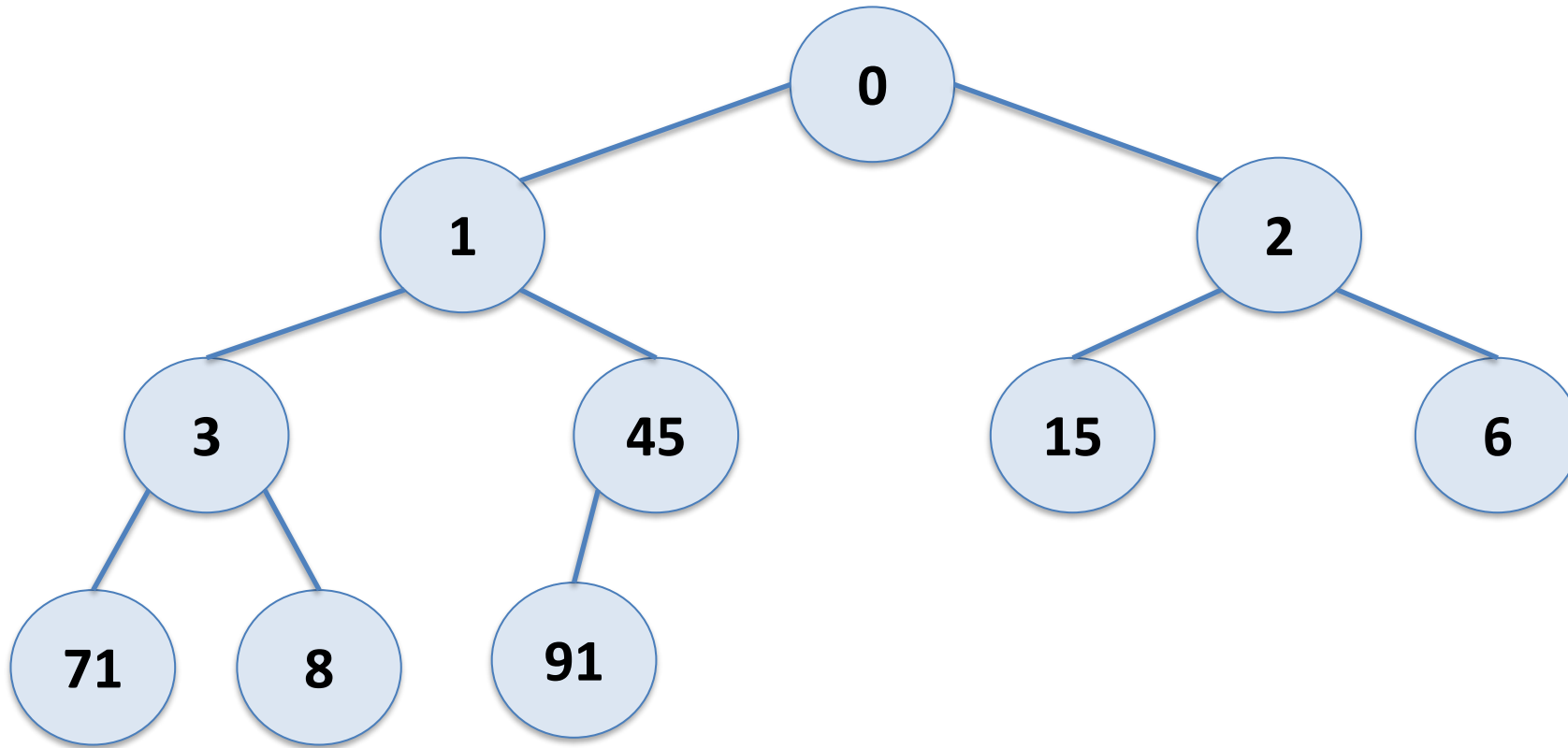
Binary Heap

Theorem 10.1 A **BinaryHeap** implements the (priority) Queue interface. Ignoring the cost of calls to `resize()`, a BinaryHeap supports the operations `add(x,p)` and `remove()` in $O(\log n)$ time per operation.

Furthermore, beginning with an empty BinaryHeap, any sequence of m `add(x,p)` and `remove()` operations results in a total of $O(m)$ time spent during all calls to `resize()`. (Hence amortized constant time.)

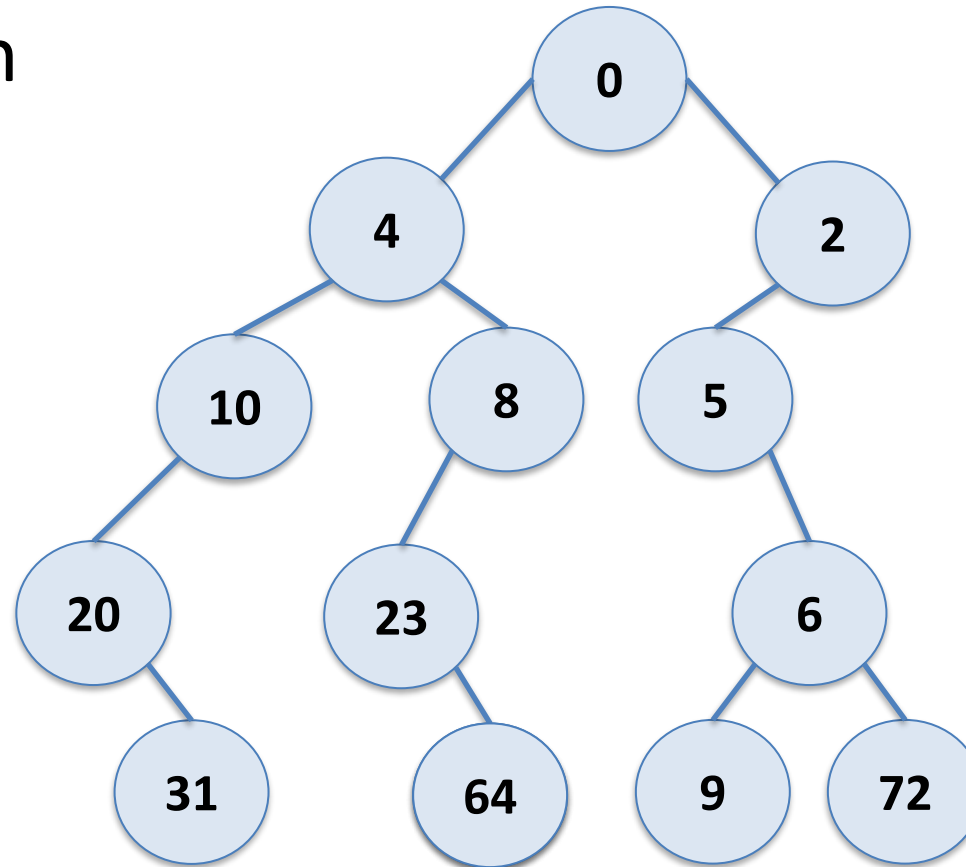
Binary Heap

- A heap-ordered binary tree
- is always a complete binary tree



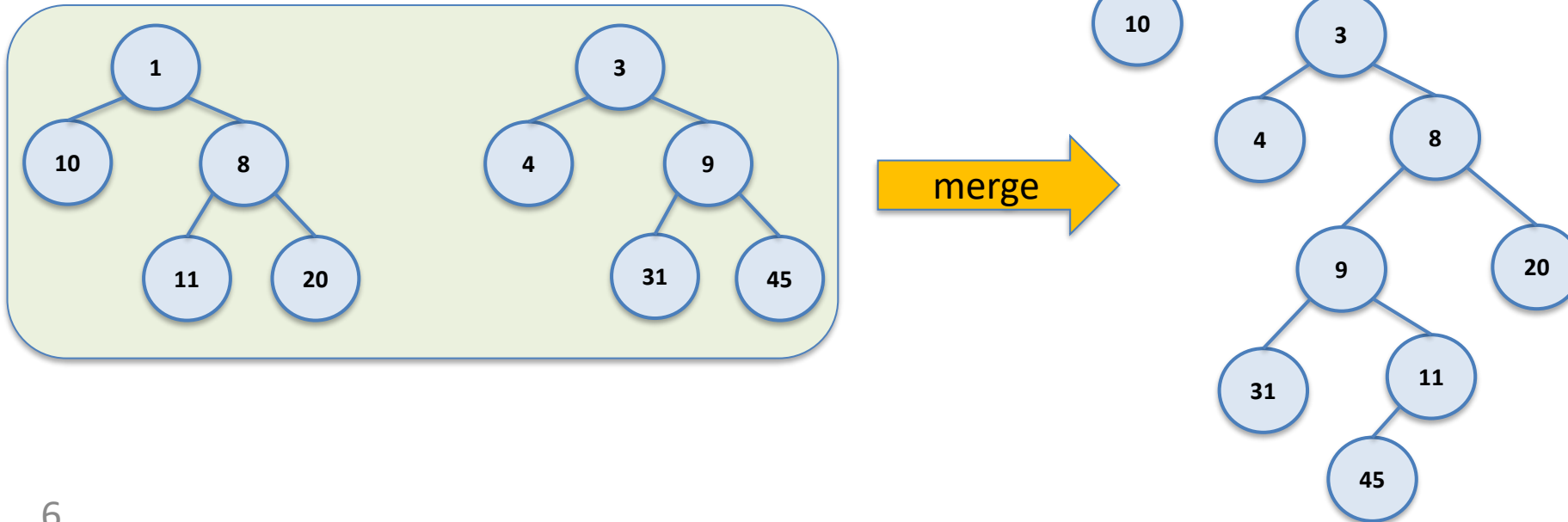
Meldable Heap

- A heap-ordered binary tree
- No restrictions on shape or size
- Uses randomization



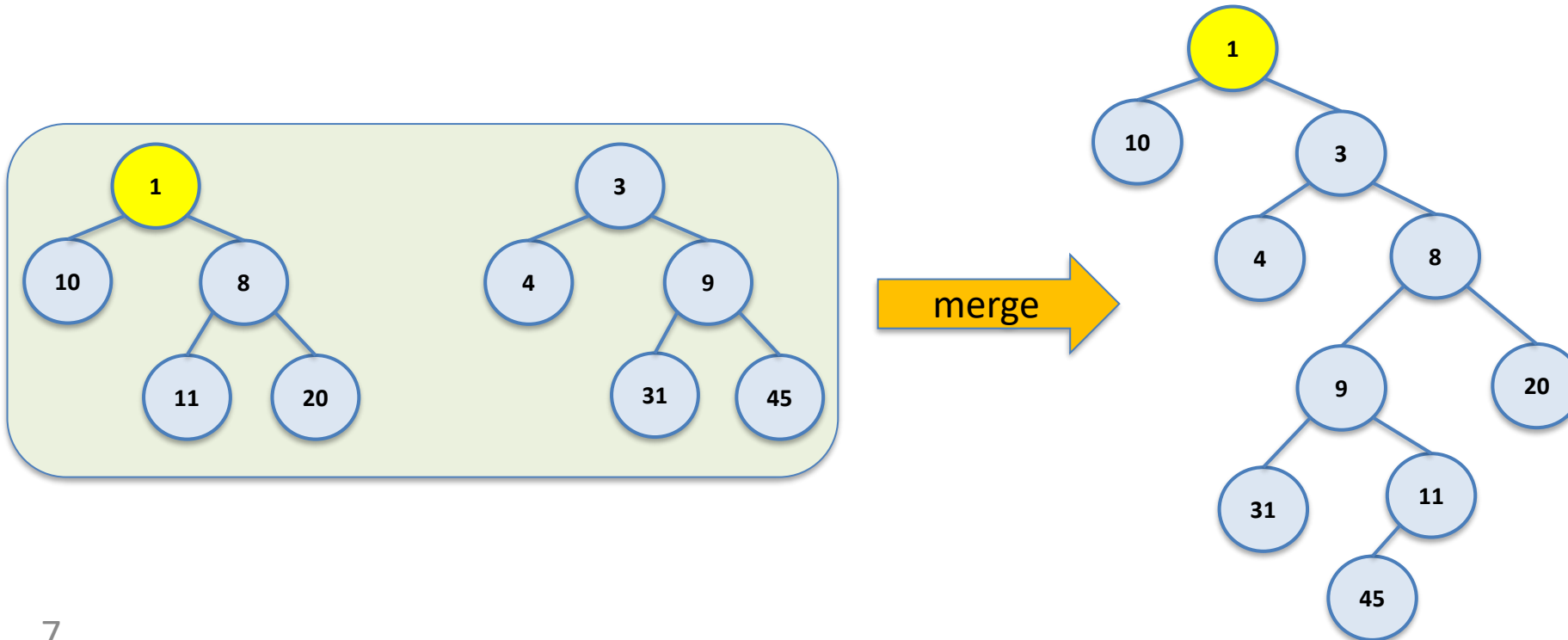
Meldable Heap

- Both add and remove involve a **merge** function
- Merge combines two meldable heaps into a single meldable heap using coin flips (randomization)



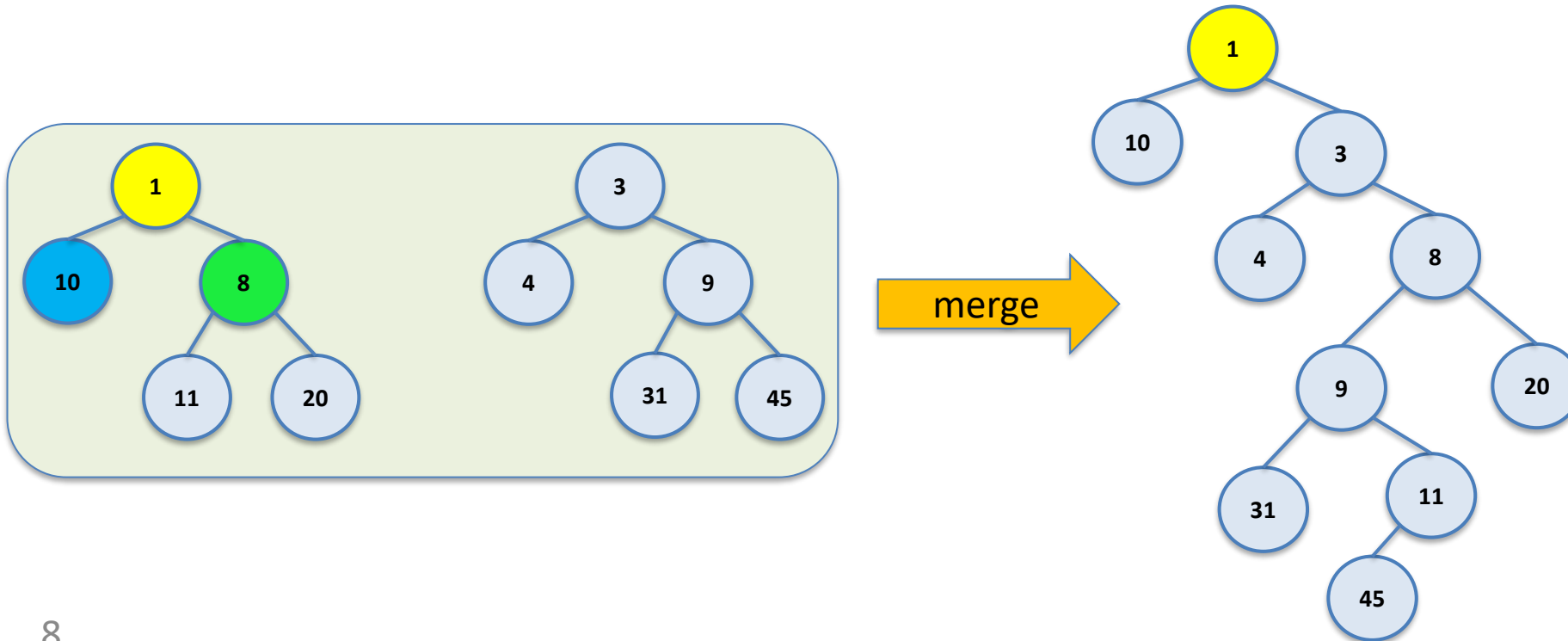
Meldable Heap

- We know that the root of the new heap must have been the root of one of the two initial heaps.
- We'll always assume the left heap has the min priority (or we'll swap them)



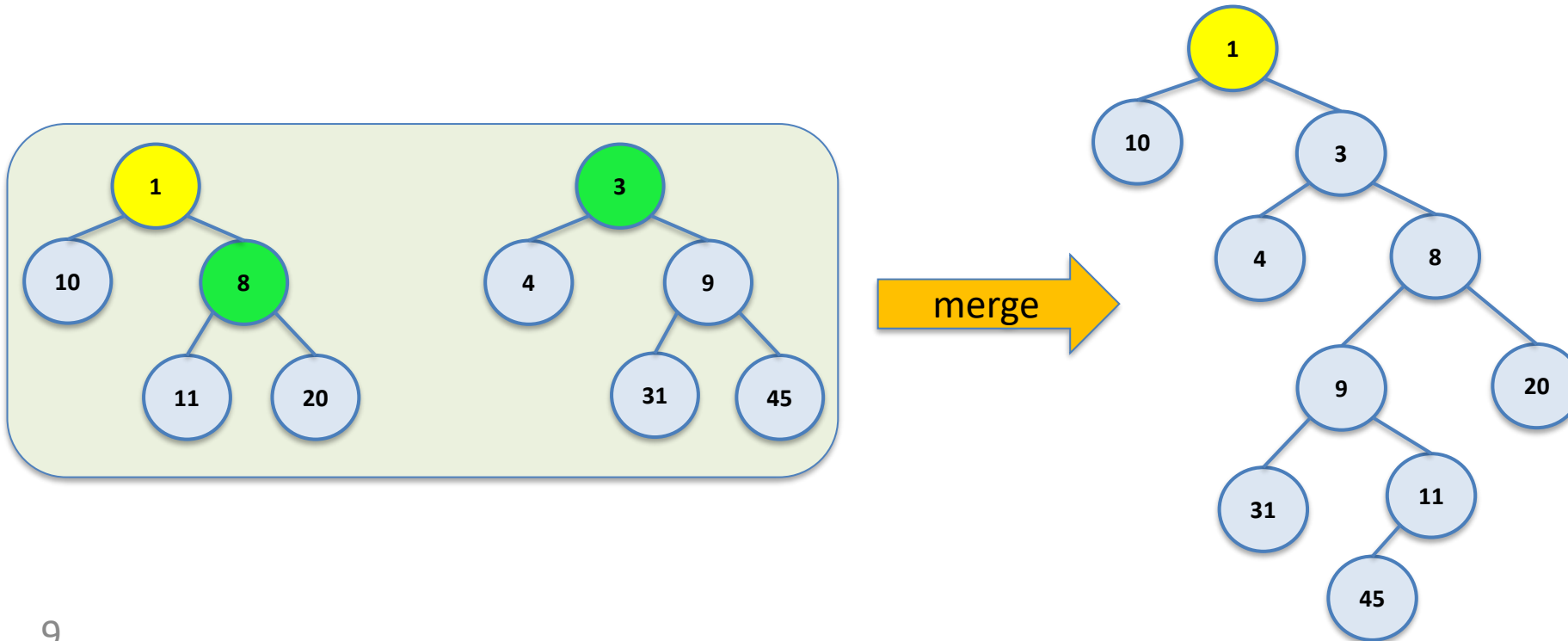
Meldable Heap

- We'll flip a coin to pick the left child or right child of the left heap and merge the right heap with it
- Merging (1) and (3) might then result in a merge of (8) and (3) [or (10) and (3)]



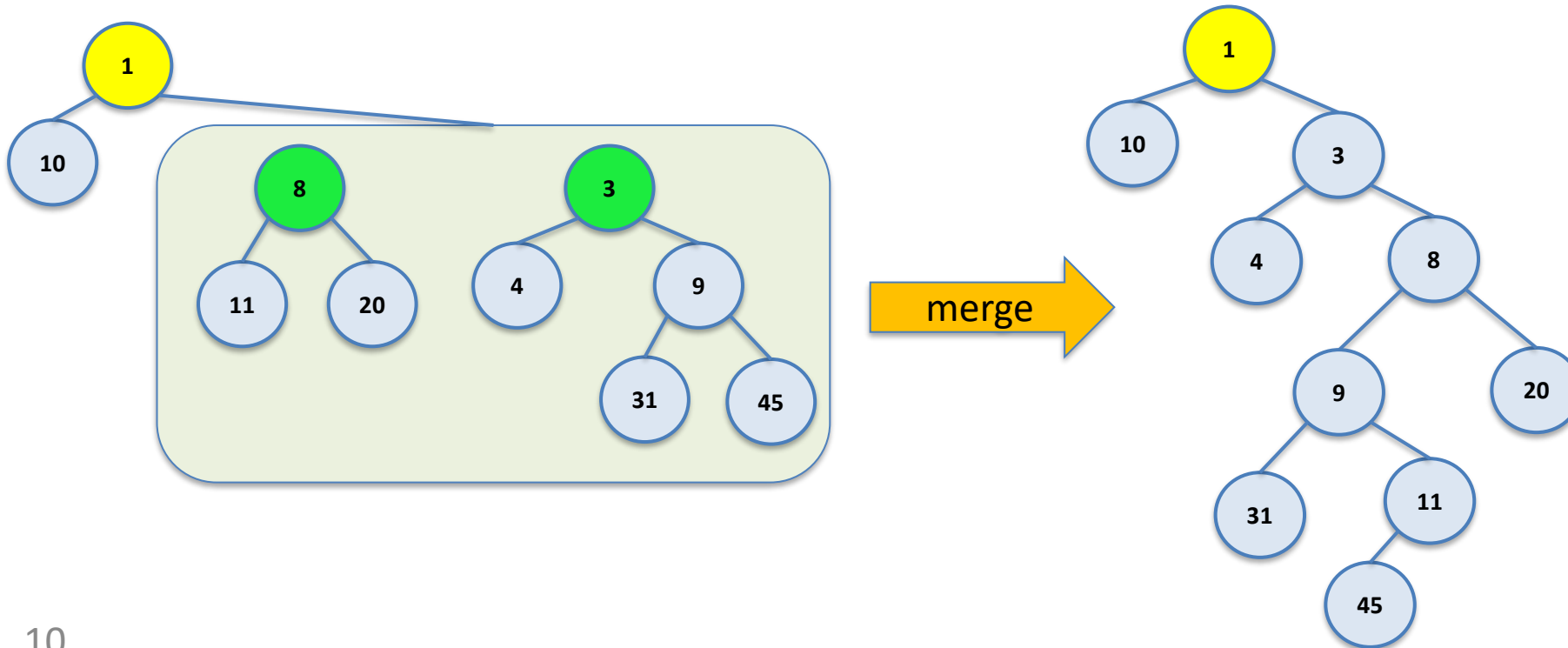
Meldable Heap

- After the coin flip in the example below, the node (10) is left alone and we'll merge (3) with (8)



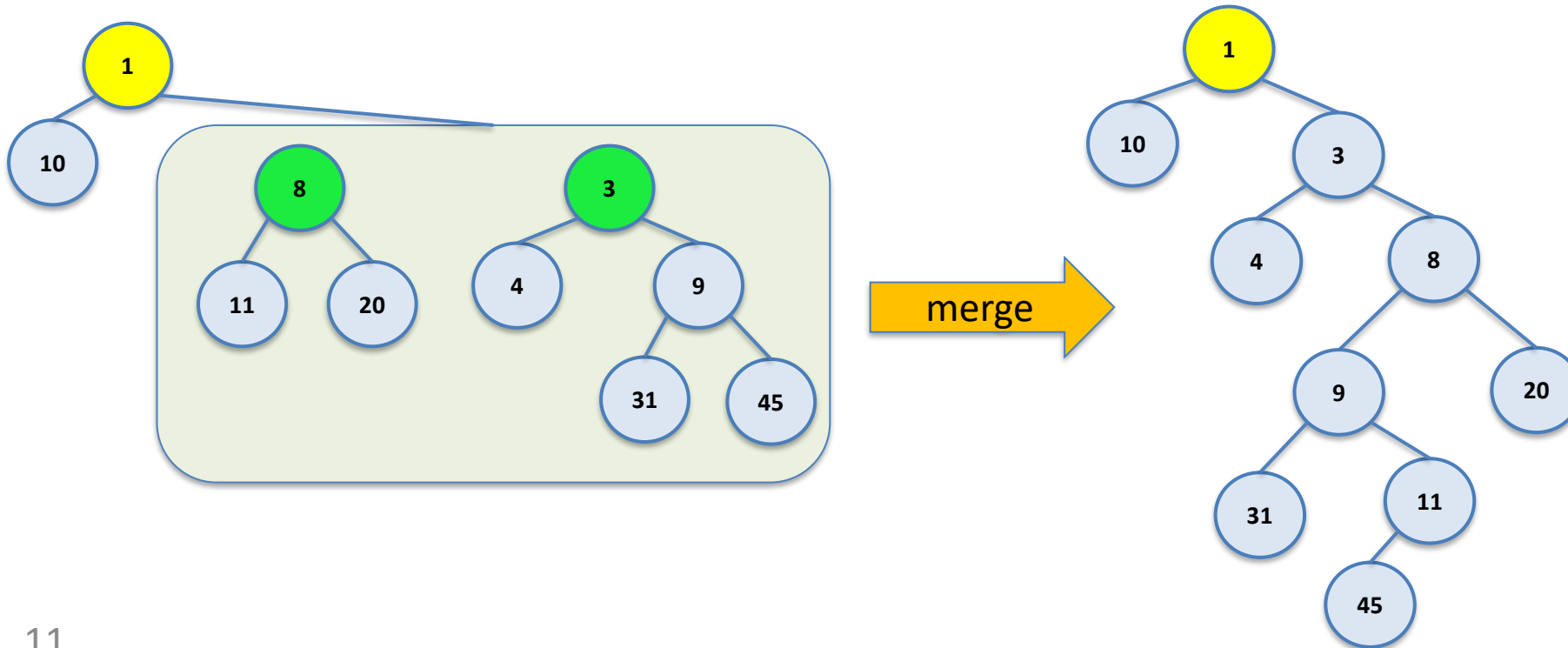
Meldable Heap

- Merge is naturally recursive.
- Merging (a) and (b) results in either merging (a) and a subtree of (b) or (b) and a subtree of (a)



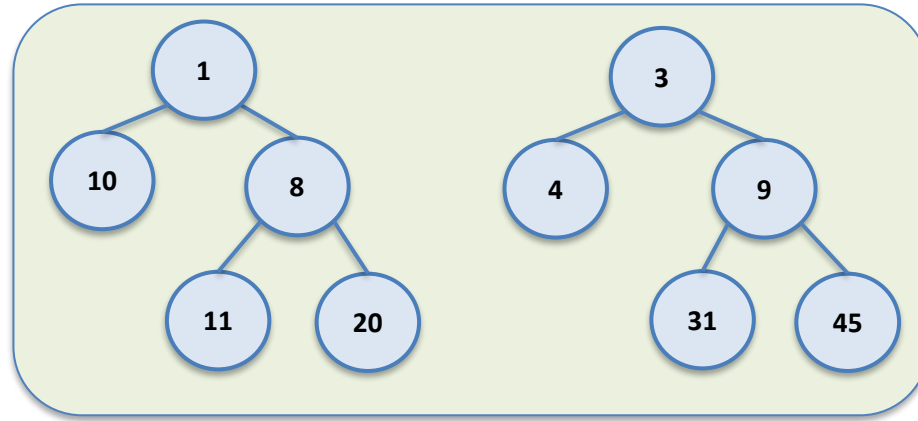
Meldable Heap

- We can merge in this recursive way until we try to merge with an empty heap (base case of recursion)
- The base case occurs when we walk off the tree



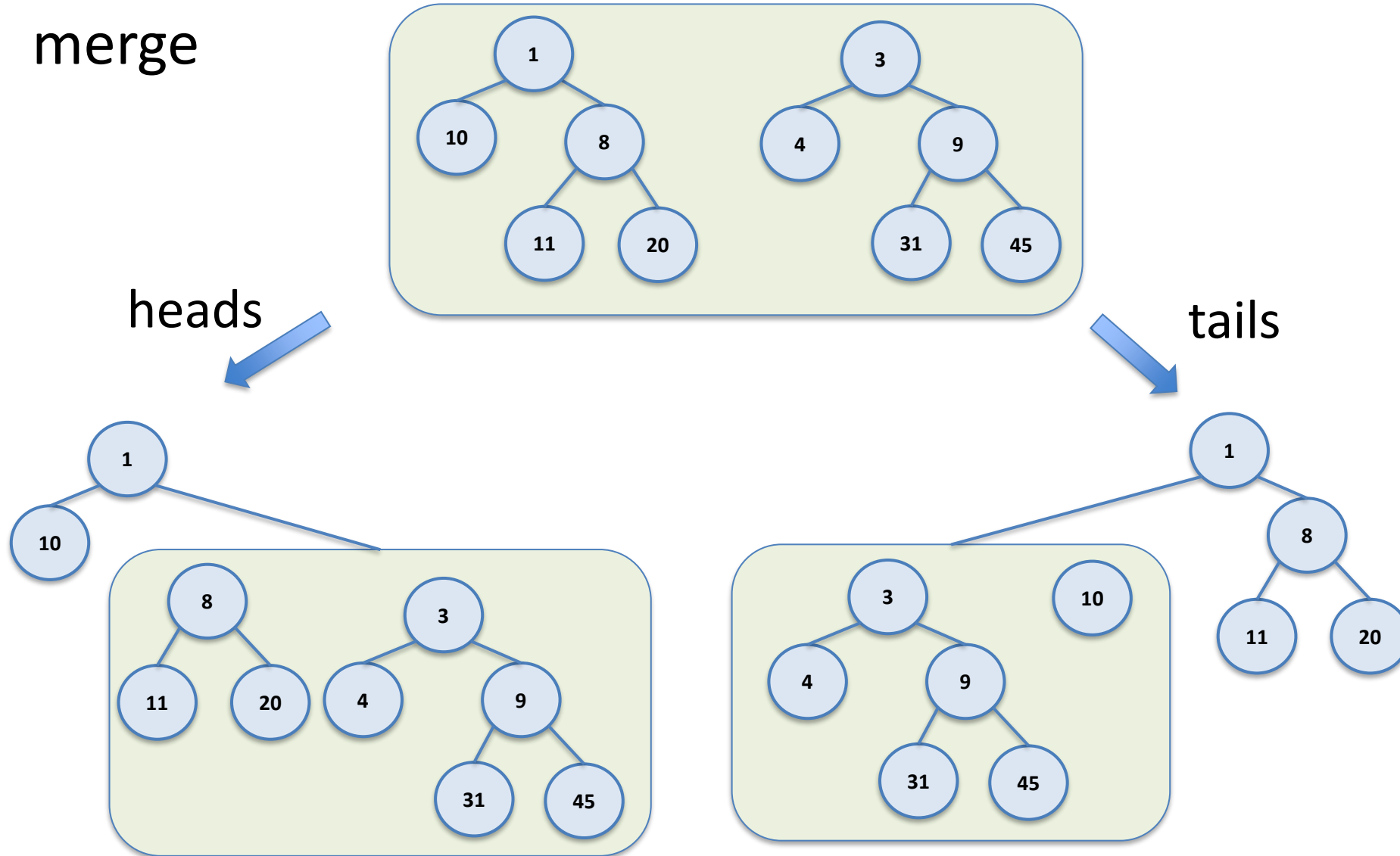
Meldable Heap

- merge



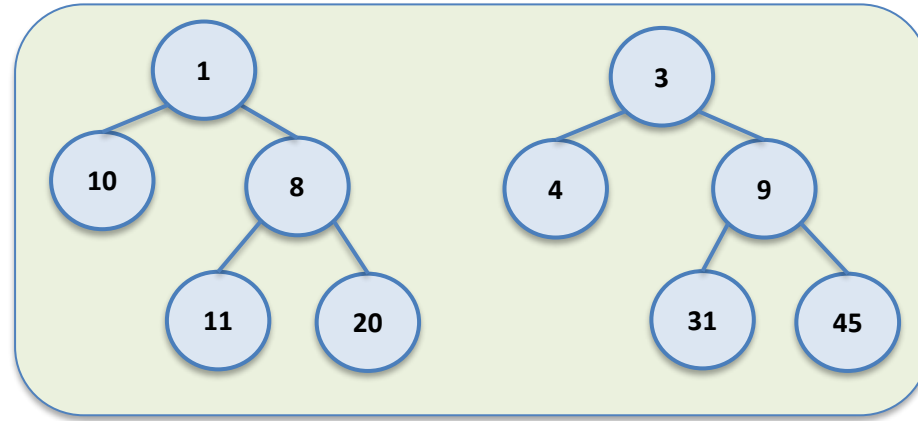
Meldable Heap

- merge

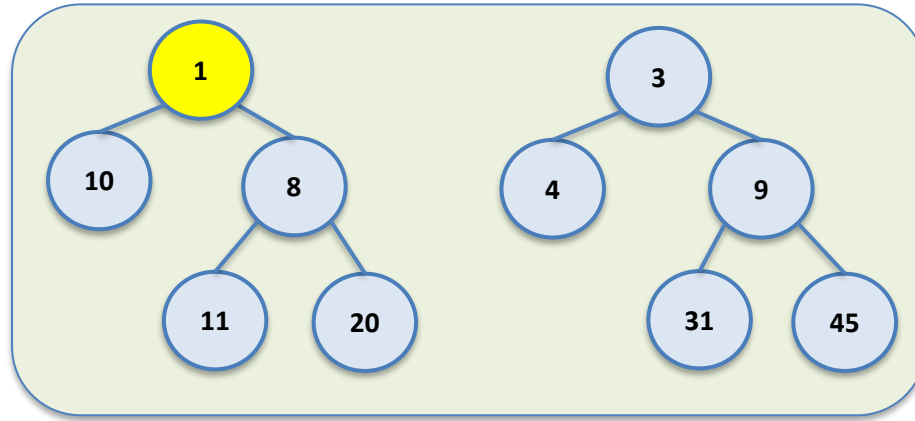


Meldable Heap

Example

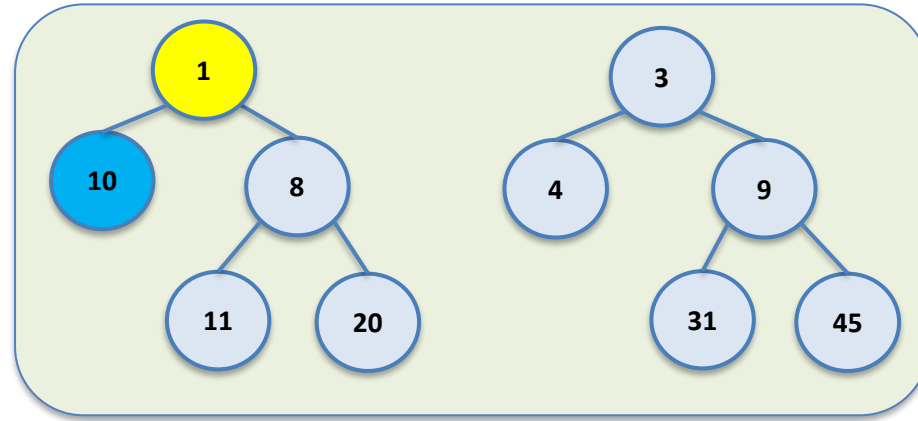


Meldable Heap



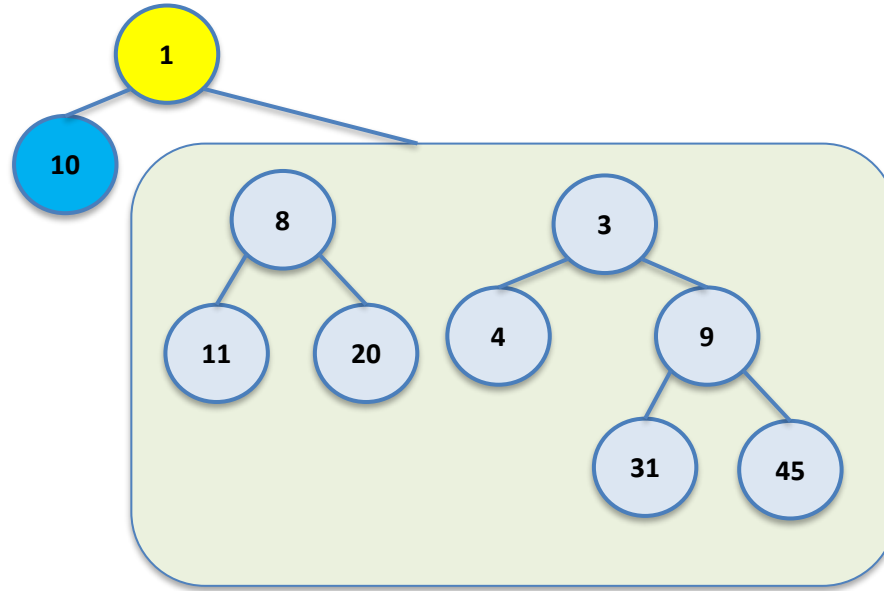
- (1) is the smallest so we use this as the root

Meldable Heap



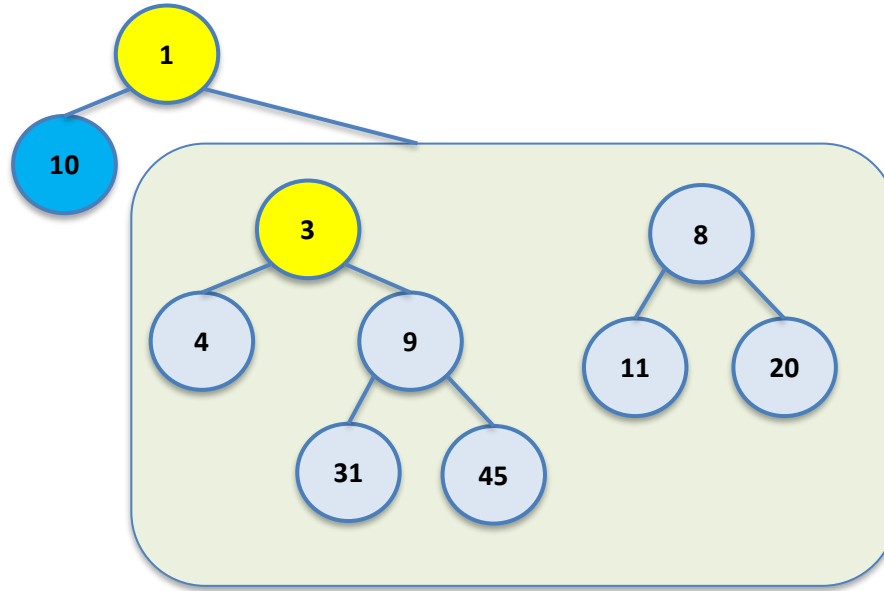
- is the smallest so we use this as the root
- flip a coin and leave left child of (1) as is
 - replace (8) with the merge of heaps (8) and (3)

Meldable Heap



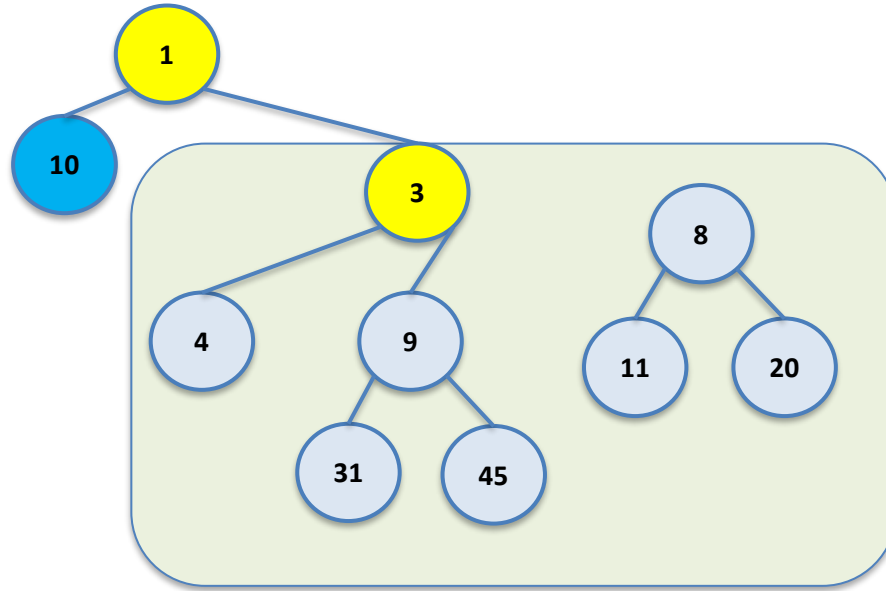
- $(8) > (3)$ so swap

Meldable Heap



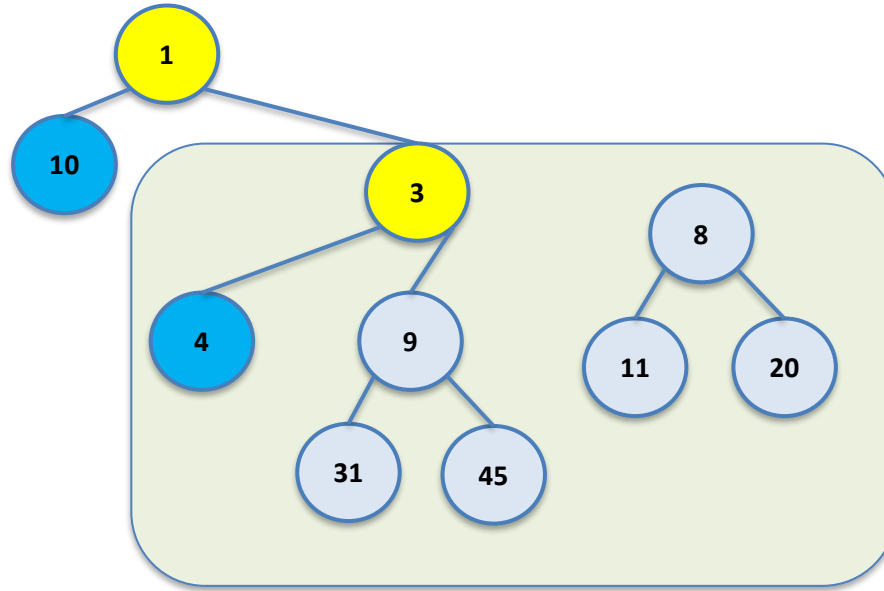
- $(8) > (3)$ so swap

Meldable Heap



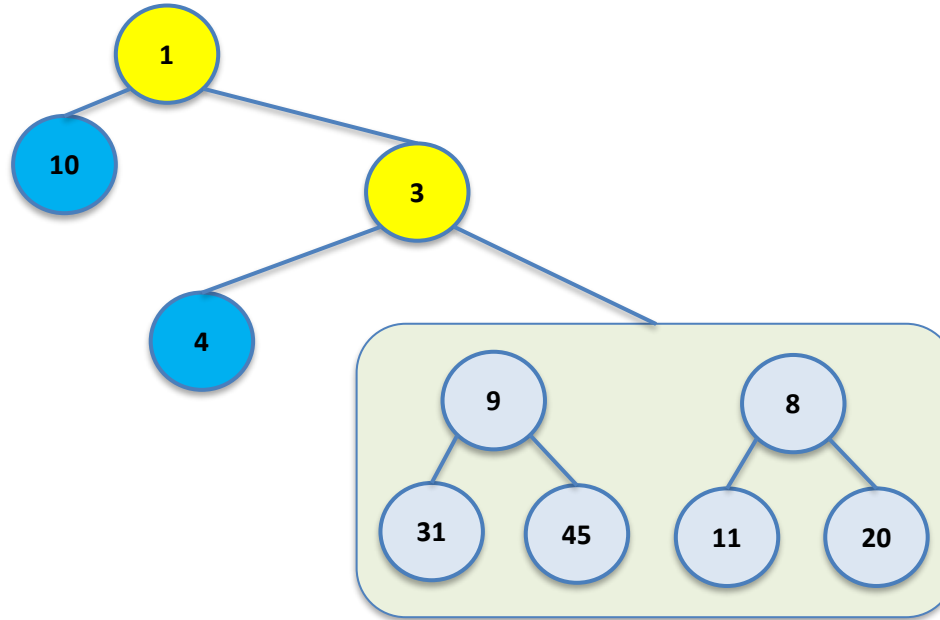
- $(8) > (3)$ so swap

Meldable Heap



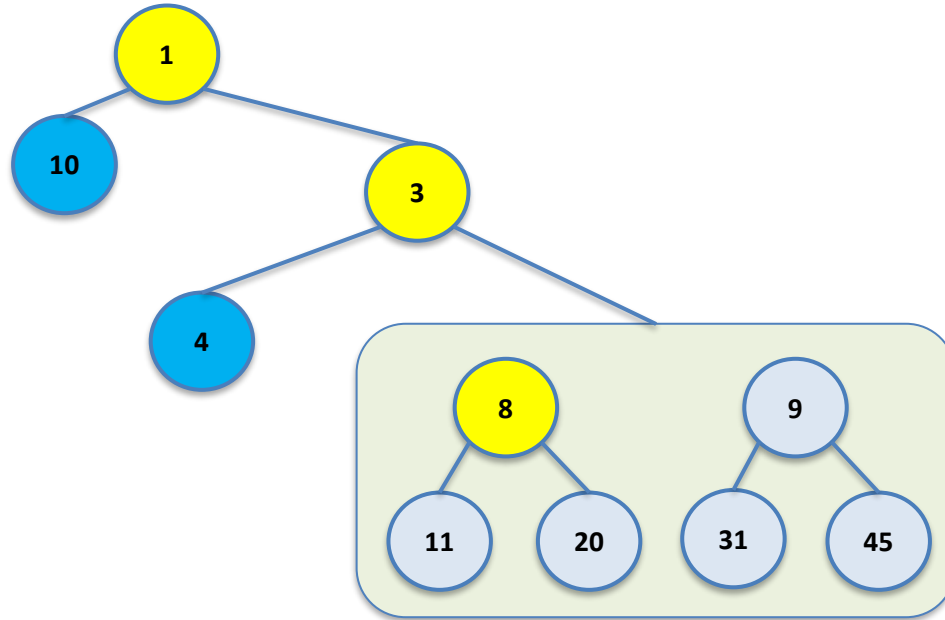
- $(8) > (3)$ so swap
- flip a coin and leave left child of (3) as is
 - replace (9) with the merge of heaps (9) and (8)

Meldable Heap



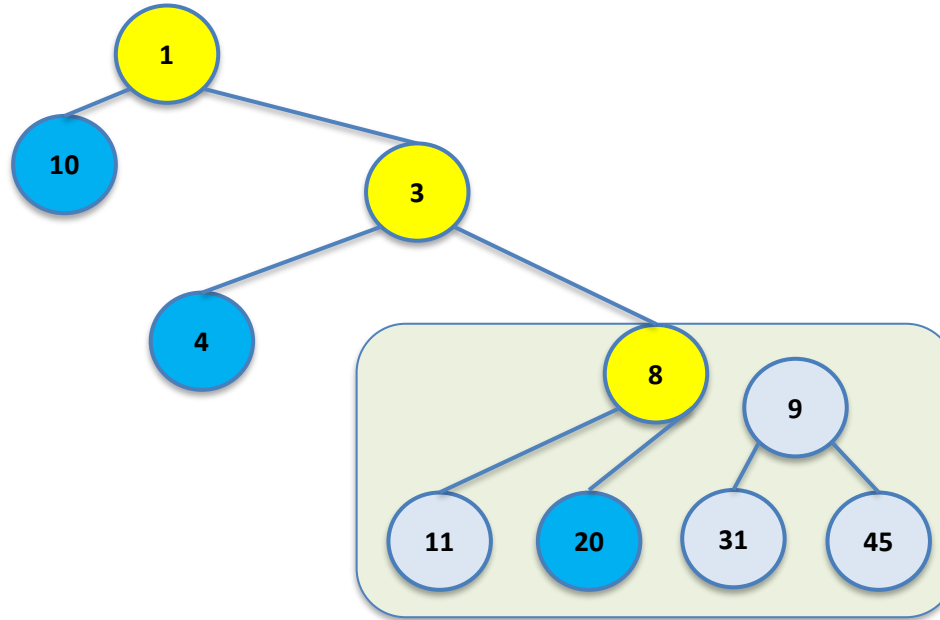
- $(9) > (8)$ so swap

Meldable Heap



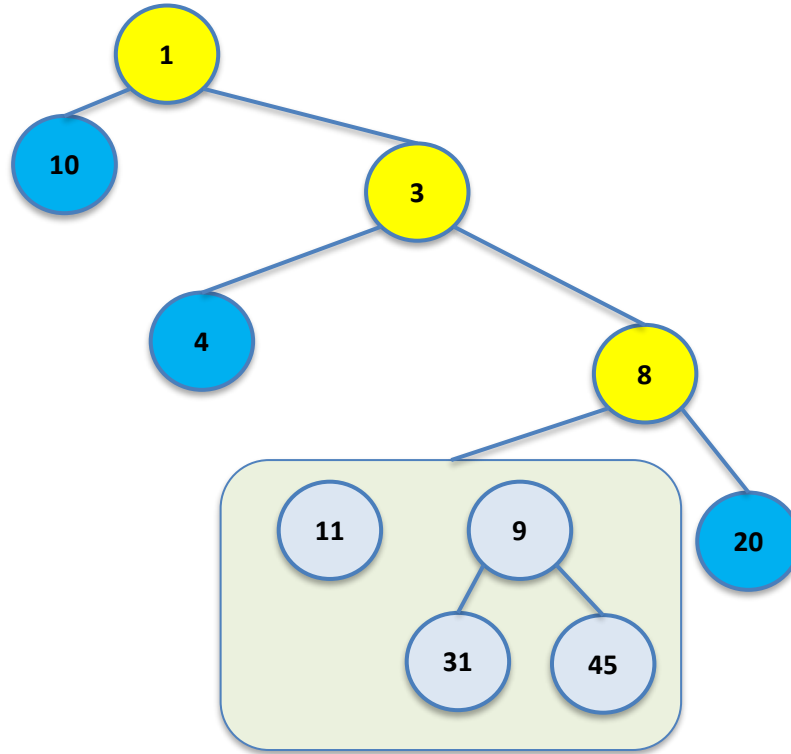
- $(9) > (8)$ so swap

Meldable Heap



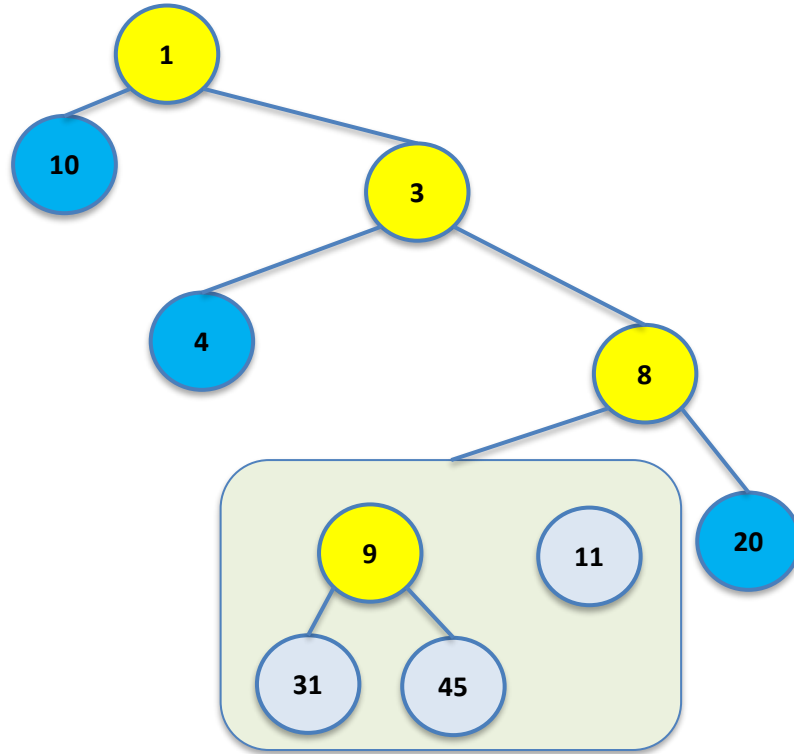
- $(9) > (8)$ so swap
- flip a coin and leave right child of (8) as is
 - replace (11) with the merge of heaps (11) and (9)

Meldable Heap



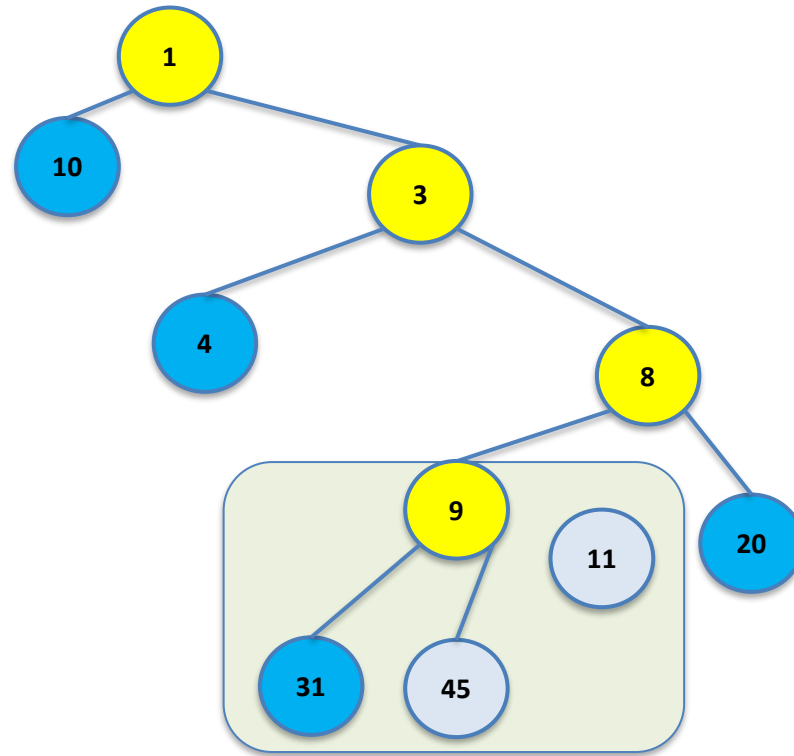
- $(11) > (9)$ so swap

Meldable Heap



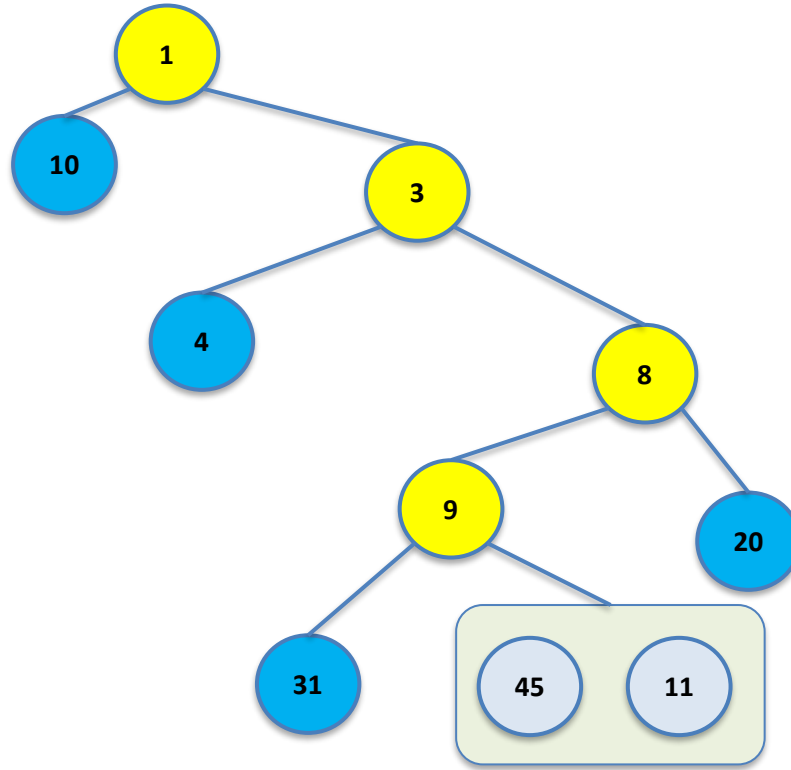
- $(11) > (9)$ so swap

Meldable Heap



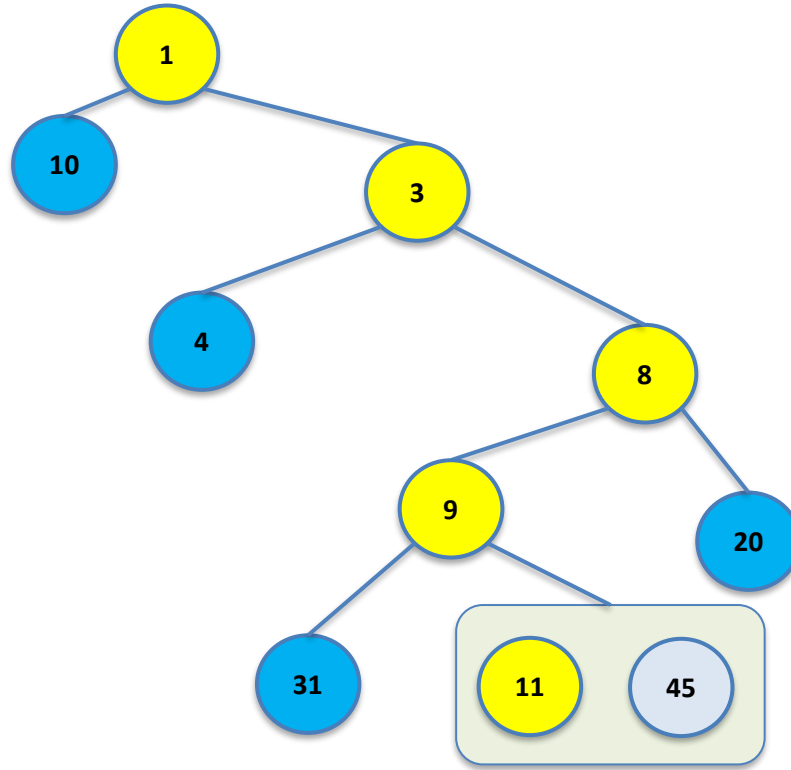
- $(11) > (9)$ so swap
- flip a coin and leave left child of (9) as is
 - replace (45) with the merge of heaps (45) and (11)

Meldable Heap



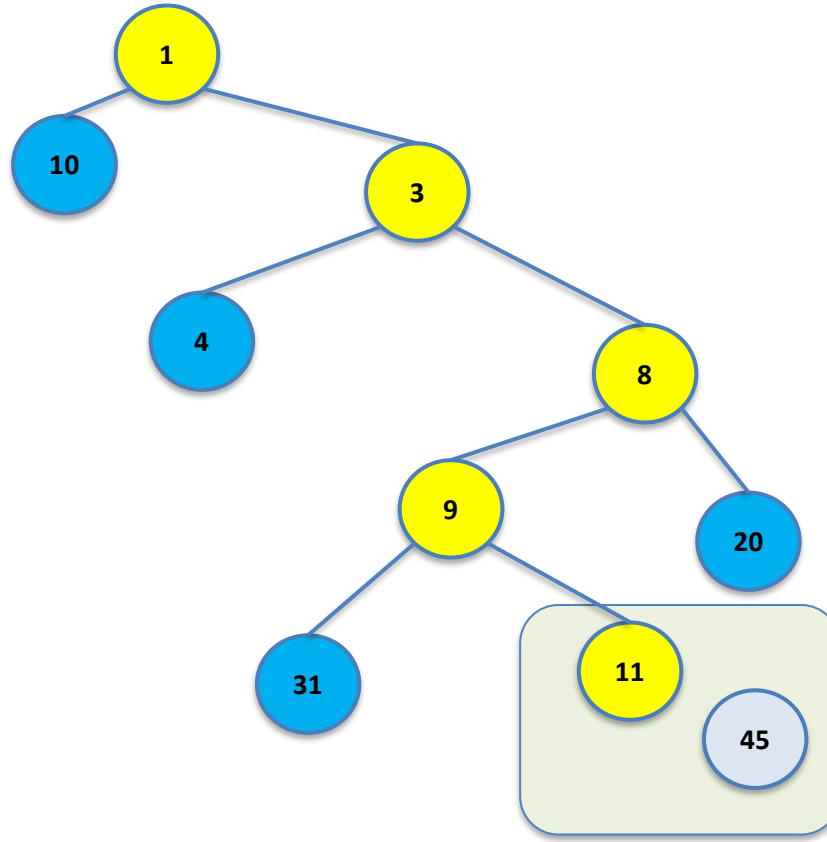
- $(45) > (11)$ so swap

Meldable Heap



- $(45) > (11)$ so swap

Meldable Heap

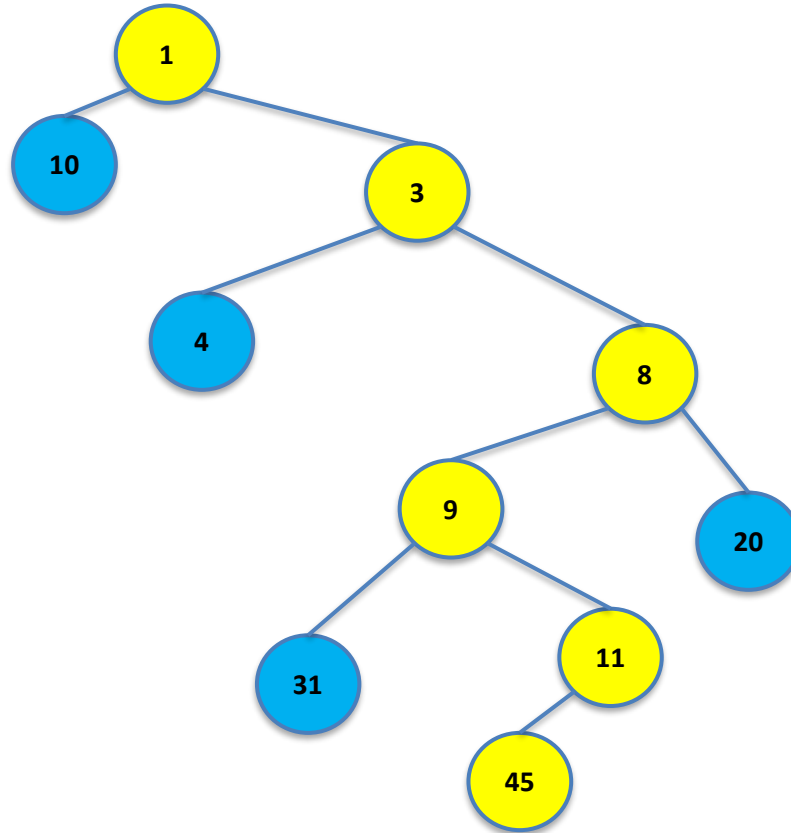


- $(45) > (11)$ so swap
- flip a coin and leave right child of (11) as is
 - merge (11) .left with heap (45)

```
graph TD; 1((1)) --- 10((10)); 1 --- 3((3)); 3 --- 4((4)); 3 --- 8((8)); 8 --- 9((9)); 8 --- 20((20)); 9 --- 31((31)); 9 --- 11((11)); 11 --- 45((45)); style 1 fill:#ffff00; style 3 fill:#ffff00; style 8 fill:#ffff00; style 9 fill:#ffff00; style 11 fill:#ffff00; style 10 fill:#00bfff; style 4 fill:#00bfff; style 20 fill:#00bfff; style 31 fill:#00bfff; style 45 fill:#add8e6; style 45_shape fill:#d9ead3,stroke:#333,stroke-width:1px; style 45_shape fill:#d9ead3,stroke:#333,stroke-width:1px; style 45_shape fill:#d9ead3,stroke:#333,stroke-width:1px;
```

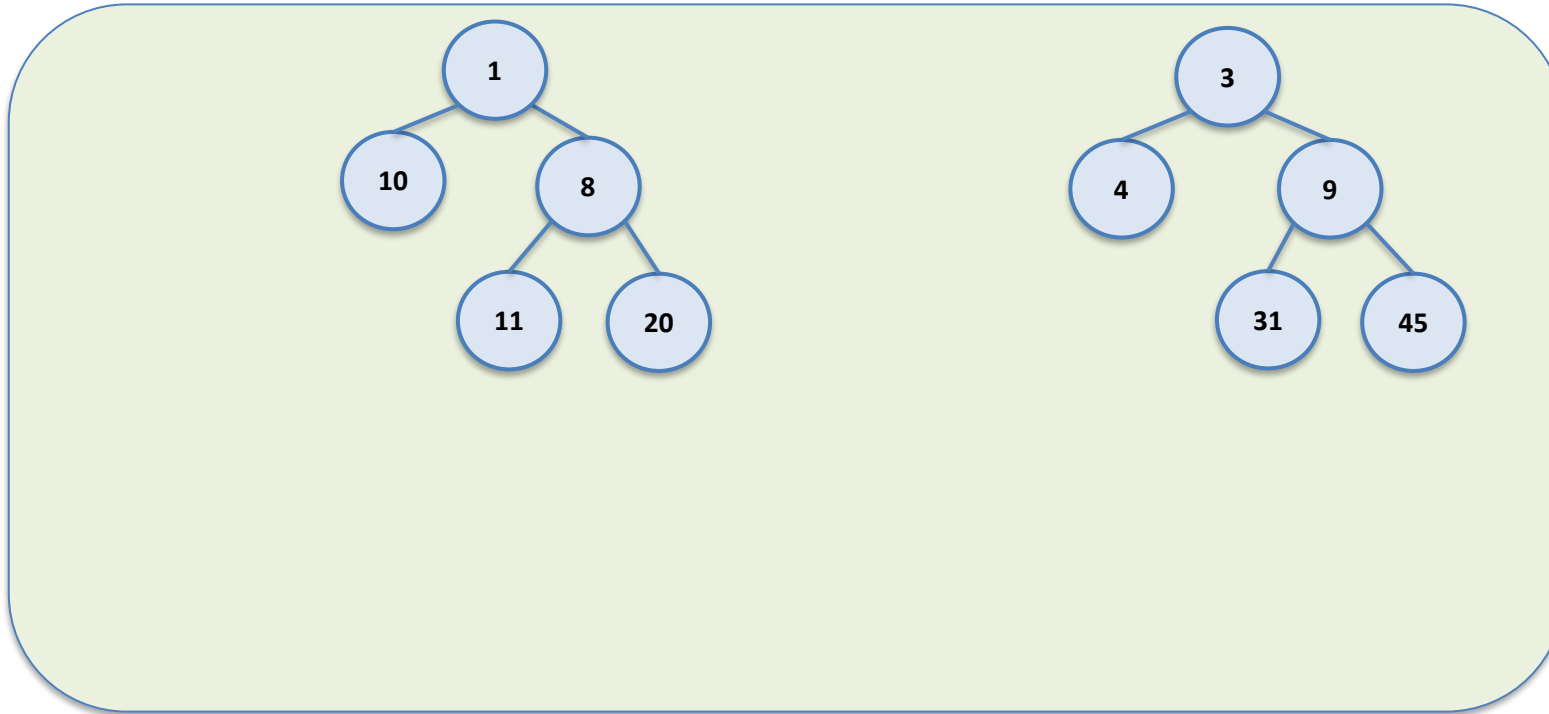
- 30

Meldable Heap



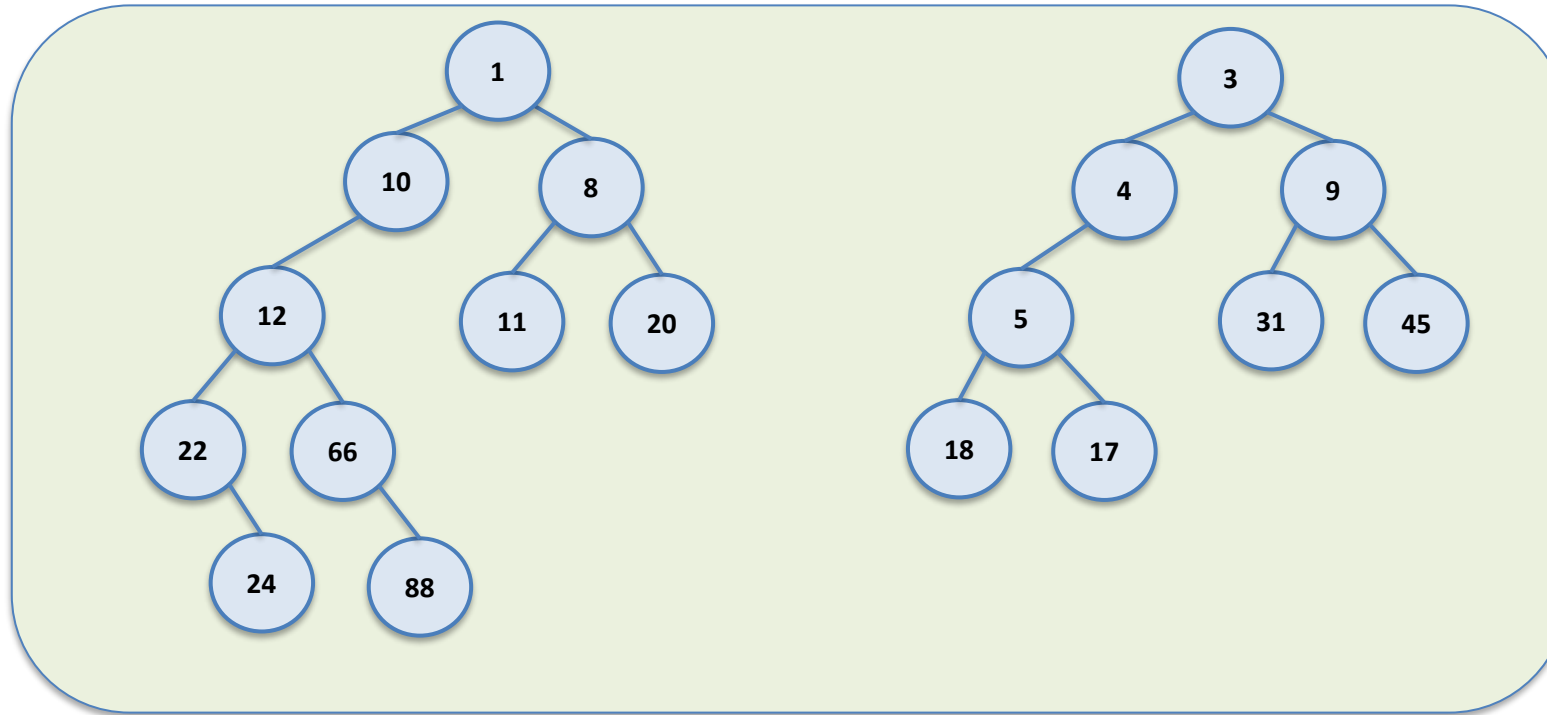
- The two initial heaps are now fully merged
- Blue nodes denotes sub-heaps that were not changed

Meldable Heap



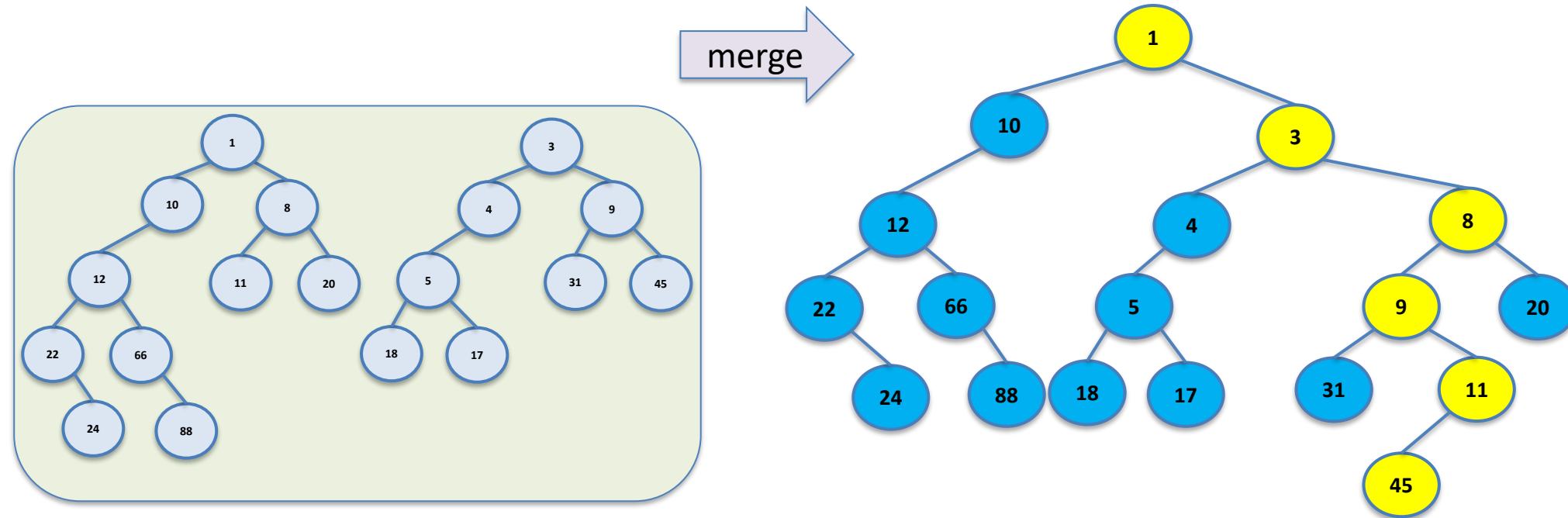
- The merged heaps grew (height) quite a bit. We were just unlucky (and the trees are small).

Meldable Heap



- consider the same coin flips when merging these heaps. What would the merged heap look like?

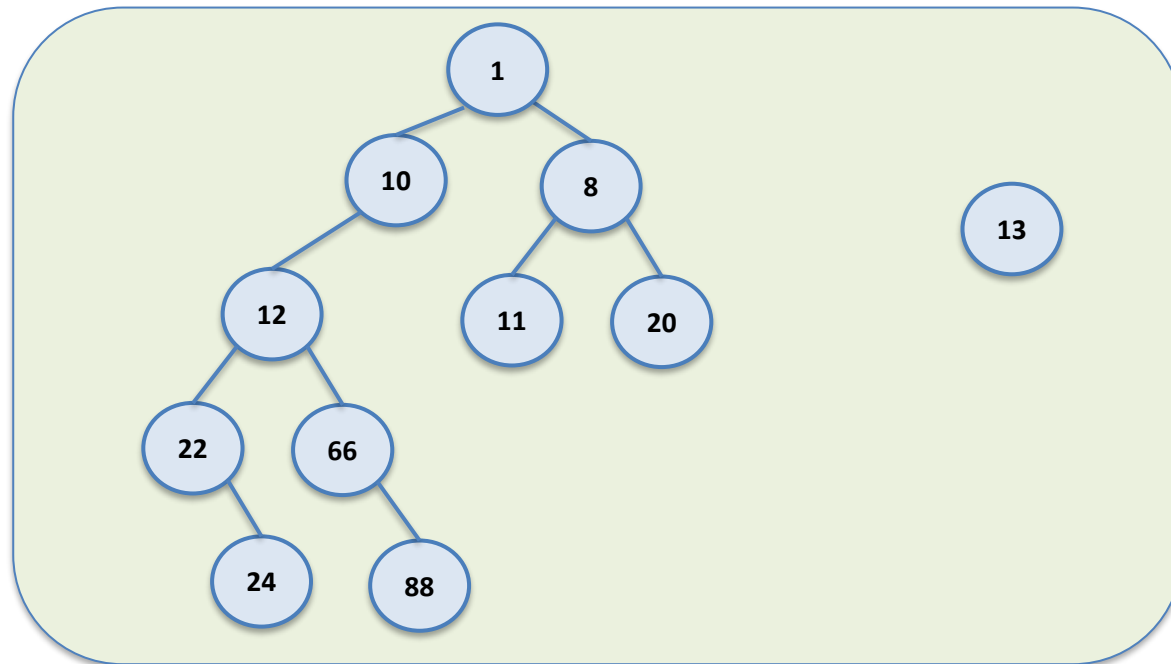
Meldable Heap



- Blue nodes are sub-heaps that were not changed during the merge. Height increased by only 1.

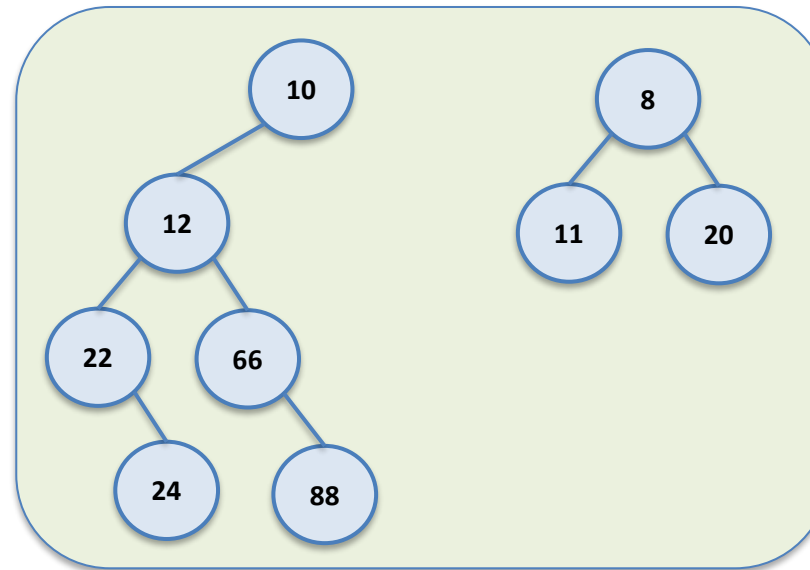
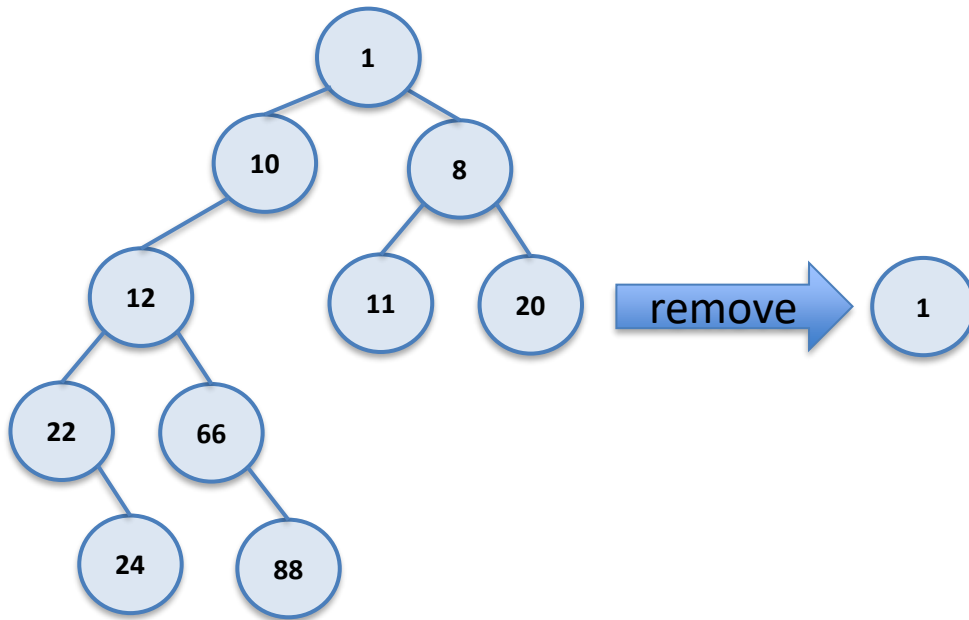
Meldable Heap

- `add(x)` to heap `h`
 - updated heap = merge of `h` and `x`
- Example: `add(13)` will merge the root of the heap with (13)



Meldable Heap

- remove() from heap h
 - min = root of h
 - updated heap = merge of root.left and root.right
 - return min



Meldable Heap

- What is the cost of `add()` and `remove()`?
- Notice that we keep merging until we try to merge with an empty heap.

We keep flipping coins and moving down the tree until we walk off the tree.

Meldable Heap

- A **random walk** in a binary tree starts at the root of the tree.
- At each step in the random walk, a coin is tossed and, depending on the result of this coin toss, the walk proceeds to the left child or to the right child of the current node.
- The walk ends when it falls off the tree (the current node becomes nil).

Lemma 10.1 The *expected* length of a random walk in a binary tree with n nodes is at most $\log(n + 1)$.

Meldable Heap

Lemma 10.1 The *expected* length of a random walk in a binary tree with n nodes is at most $\log(n + 1)$.

Meldable Heap

Theorem 10.2 A **MeldableHeap** implements the (priority) Queue interface. A MeldableHeap supports the operations `add(x)` and `remove()` in $O(\log n)$ *expected* time per operation.

Meldable Heap

Theorem 10.2 A **MeldableHeap** implements the (priority) Queue interface. A MeldableHeap supports the operations `add(x)` and `remove()` in $O(\log n)$ *expected* time per operation.

So far...

Lists

- Stack, FIFO Queue, Deque
- Array and linked list

Sorted Sets

- Skiplist, scapegoat trees, 2-4 trees, red black trees, AVL trees...

Priority Queues

- Heaps
- Array and binary tree

Sets

- ???
- Coming later...